

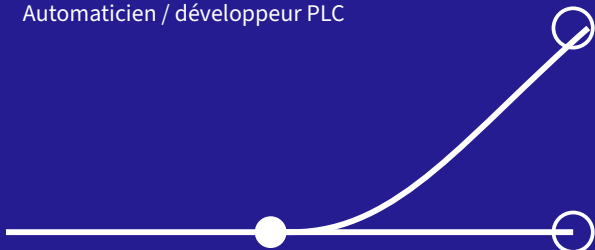
Pilotage d'un carrousel de conditionnement

De la première logique de séquencement au projet TwinCAT exécuté et testé.

Chater Bach-char

Ingénieur systèmes numériques & instrumentation

Automaticien / développeur PLC



Une architecture structurée en modes, mise à l'épreuve par des scénarios fonctionnels et des cas limites.

Objet

Présenter le carrousel réellement implémenté, exécuté sur le runtime local et testé sous TwinCAT XAE.

Périmètre

Structured Text · huit bacs
indexation pneumatique · resets et défauts

Entreprise

TQS · Tri Qualité Service

Date

4 septembre 2026

Sommaire

01 · Démarche : Contexte et objectif	3
02 · Réalisation : Système et périmètre implémenté	4
03 · Architecture TwinCAT : Modes et flux de données	5
04 · Séquencement : GRAFCET final du cycle AUTO	6
05 · Structured Text : Comptage, bacs et indexation	7
06 · Robustesse : Resets et gestion des défauts	8
07 · Essais TwinCAT : Validation fonctionnelle	9
08 · Retour d'essais : Cas limites détectés et corrigés	10
09 · Périmètre maîtrisé : Limites et évolutions possibles	11
Annexe A · TwinCAT 3 : Types et interface du bloc	12
10 · Synthèse : Conclusion	13

01 à 02

contexte, système étudié et périmètre réellement implémenté;

03 à 06

architecture TwinCAT, GRAFCET et traitements ST;

07 à 08

essais fonctionnels et cas limites corrigés;

09 à 10

limites, évolutions et synthèse du travail réalisé;

Annexe

types, interface et extraits du source consolidé.

01 · DÉMARCHÉ

Contexte et objectif

Conserver la trace de la première approche, puis présenter le projet réellement développé et testé.

Positionnement

Lors du test, une version volontairement simplifiée a été développée afin de présenter rapidement le principe de fonctionnement. Le projet a ensuite été repris sous TwinCAT pour approfondir l'architecture, les huit bacs, les resets et plusieurs cas limites.

Objectif du rapport

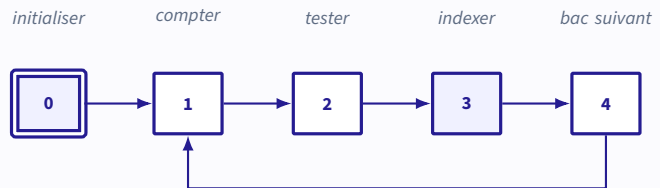
Le document distingue clairement la **solution de principe montrée pendant l'entretien** de la **version approfondie exécutée sur le runtime local**. Il ne présente pas comme réalisées les fonctions qui restent hors du projet.

Fil conducteur

1. poser le cycle nominal;
2. isoler les modes et les états;
3. traduire la séquence en ST;
4. provoquer des cas limites;
5. corriger puis rejouer les essais.

Première lecture fonctionnelle

Le GRAFCET initial reste une trace utile : il montre le raisonnement présenté à l'automaticien sans lui attribuer le niveau de couverture du projet final.



Ce qui a changé ensuite

La reprise sous TwinCAT remplace cette séquence linéaire par deux niveaux : un mode global INIT / AUTO / DEFAULT, puis une machine d'états dédiée au cycle AUTO.

8

bacs mémorisés

4

cycles / index

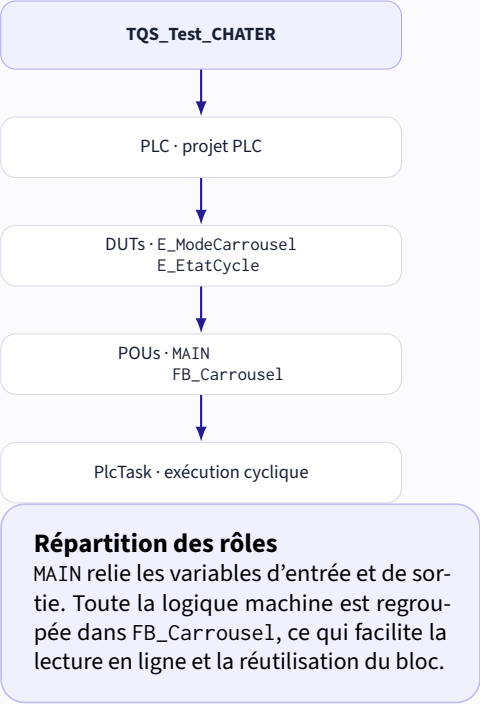
10 s

reset global

Système et périmètre implémenté

Un bloc fonction unique porte la logique métier; le programme principal reste volontairement léger.

Structure du projet XAE



Fonctions présentes dans la version réalisée

Fonction	Comportement implémenté
Référencement	attente du capteur inductif, puis affectation du bac 1
Comptage	front montant du laser avec R_TRIG
Mémoire	huit compteurs dans aPieceBac[1..8]
Indexation	quatre cycles complets sortie + rentrée du vérin
Temporisation	temps de mouvement réglable par tVerin
Bouclage	retour logique du bac 8 vers le bac 1
Bac plein	attente opérateur dans ATTENTE_BAC
Reset	huit resets individuels; reset global après 10 s
Défaut	paramètres invalides, commandes coupées, reprise par INIT

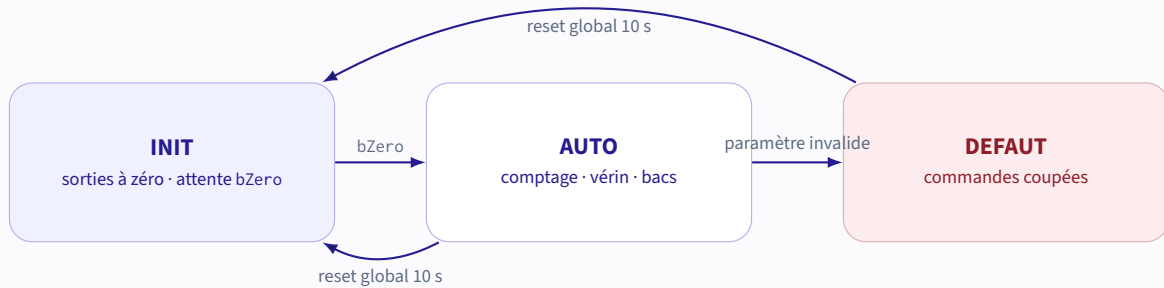
Interprétation retenue

Le terme « 4 impulsions » a été traduit par quatre cycles complets du vérin : sortie temporisée, rentrée temporisée, puis incrément de nImp.

03 · ARCHITECTURE TWINCAT

Modes et flux de données

Trois modes au niveau machine ; une seconde machine d'états pour détailler le cycle automatique.



DUTs retenus

E_ModeCarrousel

INIT

AUTO

DEFAULT

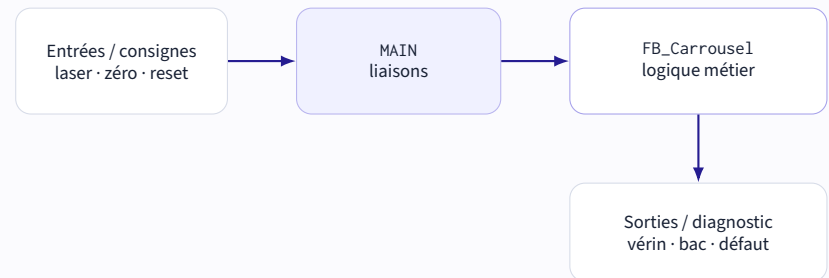
E_EtatCycle

ATTENTE, COMPTAGE, BAC_PLEIN,
VERIN_SORTIE, VERIN_RENTREE,
BAC_SUIVANT, ATTENTE_BAC.

Priorité globale

Le reset long est traité avant le CASE des modes. Il remet les compteurs et les sorties à zéro, acquitte le défaut, puis impose un retour par INIT.

Flux du programme

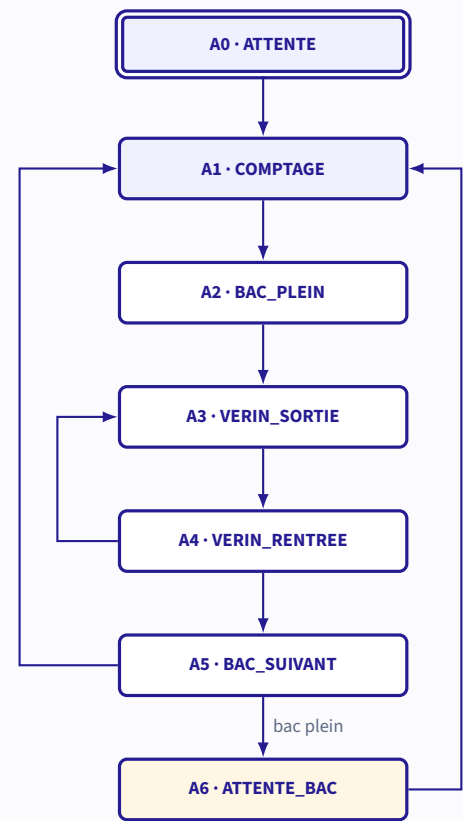


Règles de fonctionnement

- la logique de bac n'est exécutée qu'en AUTO;
- l'index nBac est contrôlé avant accès au tableau;
- les deux commandes du vérin ne sont jamais vraies simultanément;
- le changement de bac intervient après le quatrième cycle complet;
- un bac déjà plein bloque le cycle jusqu'à son reset.

GRAFCET final du cycle AUTO

Le diagramme reprend les états et les transitions réellement présents dans le bloc fonction.



Réceptivités principales

Transition	Condition
A0 → A1	entrée dans le mode AUTO
A1 → A2	aPieceBac[nBac] >= nPieceBac
A2 → A3	préparation de l'indexation
A3 → A4	temporisation de sortie écoulée
A4 → A3	temporisation de rentrée écoulée et nImp < 4
A4 → A5	temporisation écoulée et nImp >= 4
A5 → A1	nouveau bac sous la consigne
A5 → A6	nouveau bac déjà plein
A6 → A1	compteur du bac remis sous la consigne

Lecture du cycle vérin

Une unité de nImp correspond à une sortie suivie d'une rentrée. Le plateau ne passe au bac suivant qu'après quatre cycles complets.

Référencement

INIT exploite le capteur inductif comme référence. La stratégie mécanique de recherche du zéro n'étant pas définie, la version réalisée considère bZero = TRUE comme condition de validation du référencement.

05 · STRUCTURED TEXT

Comptage, bacs et indexation

Des traitements courts, directement observables en ligne dans le bloc fonction.

Front du capteur laser

```
1 fbLaser(CLK := bLaser);
2
3 IF fbLaser.Q THEN
4   aPieceBac[nBac] :=
5     aPieceBac[nBac] + 1;
6 END_IF
```

R_TRIG évite d'incrémenter plusieurs fois la même pièce lorsque le signal reste actif pendant plusieurs cycles PLC. Il ne remplace pas un filtrage matériel ou un traitement antiparasite.

Données par bac

aPieceBac : ARRAY[1..8] OF INT
Une valeur est conservée pour chaque bac.
Seule la case correspondant à nBac est incrémentée.

Noms retenus

bLaser, bZero, nBac, nImp, tVerin et
aResetBac restent courts et lisibles pour
une mise au point en atelier.

Séquence pneumatique réelle

```
1 E_EtatCycle.VERIN_SORTIE:
2   bVerinAv := TRUE;
3   bVerinAr := FALSE;
4   fbTempoVerin(IN := TRUE,
5                 PT := tVerin);
6
7 IF fbTempoVerin.Q THEN
8   bVerinAv := FALSE;
9   fbTempoVerin(IN := FALSE);
10  eEtatCycle :=
11    E_EtatCycle.VERIN_RENTREE;
12 END_IF
13
14 E_EtatCycle.VERIN_RENTREE:
15   bVerinAv := FALSE;
16   bVerinAr := TRUE;
17   fbTempoVerin(IN := TRUE,
18                 PT := tVerin);
19
20 IF fbTempoVerin.Q THEN
21   bVerinAr := FALSE;
22   fbTempoVerin(IN := FALSE);
23   nImp := nImp + 1;
24
25   IF nImp >= 4 THEN
26     eEtatCycle :=
27       E_EtatCycle.BAC_SUIVANT;
28   ELSE
29     eEtatCycle :=
30       E_EtatCycle.VERIN_SORTIE;
31   END_IF
32 END_IF
```

06 · ROBUSTESSE

Resets et gestion des défauts

Les commandes sont neutralisées avant toute reprise ; le reset long sert aussi d'acquiescement.

Reset individuel et global

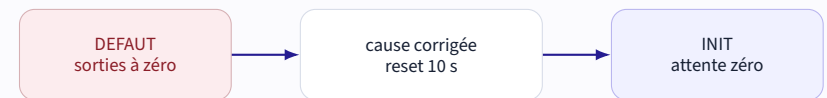
```
1 bResetGlobal := FALSE;
2
3 FOR i := 1 TO 8 DO
4   IF aResetBac[i] THEN
5     aPieceBac[i] := 0; // reset bac
6   END_IF
7
8   aTempoReset[i](
9     IN := aResetBac[i],
10    PT := T#10S);
11
12   IF aTempoReset[i].Q THEN
13     bResetGlobal := TRUE;
14   END_IF
15 END_FOR
```

Effet du reset global

Les huit compteurs, les deux commandes et nImp sont remis à zéro. Le défaut est acquitté, le timer vérin est réinitialisé et le mode repasse à INIT.

Conditions de défaut présentes

Condition	Réaction
nPieceBac <= 0	passage en DEFAUT
tVerin <= T#0MS	passage en DEFAUT
nBac hors 1 à 8	blocage avant accès au tableau
état cycle incohérent	branche ELSE, puis DEFAUT
mode DEFAUT	sorties coupées, timer réinitialisé
reset global 10 s	acquiescement et retour par INIT



Portée réelle

Cette version gère des incohérences logicielles et de paramétrage. Elle ne réalise pas encore de diagnostic mécanique détaillé ni de fonction Safety.

Validation fonctionnelle

Le projet a été compilé dans XAE, téléchargé sur le runtime local et observé en ligne.

0

erreur au Rebuild

11

scénarios rejoués

3

cas limites corrigés

Essai	Résultat observé	Statut
Détection du point zéro	INIT → nBac = 1 → AUTO	OK
Front du capteur laser	une seule incrémentation pour FALSE → TRUE → FALSE	OK
Consigne atteinte	passage de COMPTAGE à BAC_PLEIN	OK
Quatre cycles vérin	alternance sortie / rentrée, puis BAC_SUIVANT	OK
Retour 8 vers 1	bouclage logique correct	OK
Bac suivant déjà plein	maintien dans ATTENTE_BAC	OK
Reset individuel	seul le compteur visé passe à zéro	OK
Reset global 10 s	remise à zéro des huit compteurs	OK
nPieceBac = 0	passage en DEFAUT, vérin arrêté	OK
tVerin = T#0MS	passage en DEFAUT, vérin arrêté	OK
Acquittement après correction	reset 10 s, retour INIT, puis AUTO au zéro	OK

Traçabilité de la version

Les essais ont été réalisés sur la logique fonctionnelle précédant immédiatement la dernière mise au propre. Le comportement des états et les corrections décrites sont identiques; un Rebuild du source consolidé reste la vérification finale avant archivage comme version de référence.

Cas limites détectés et corrigés

La simulation a fait apparaître des comportements qui n'étaient pas visibles dans le cycle nominal.

1 • Consigne de pièces égale à zéro

Symptôme	$0 \geq 0$ rendait chaque bac plein immédiatement et lançait une rotation continue.
Correction	contrôle de $nPieceBac \leq 0$ avant l'exécution du cycle AUTO.
Résultat	passage en DEFAUT et arrêt des deux commandes vérin.

2 • Huit bacs déjà pleins

Symptôme	après le retour 8 vers 1, le programme pouvait continuer à parcourir les bacs sans intervention opérateur.
Correction	ajout de l'état ATTENTE_BAC lorsque le compteur du nouveau bac a déjà atteint la consigne.
Résultat	arrêt sur le bac concerné, reset opérateur, puis reprise du comptage.

3 • Temps vérin égal à zéro

Symptôme	le TON terminait immédiatement et les états pouvaient s'enchaîner à la cadence du cycle PLC.
Correction	contrôle de $tVerin \leq T\#0MS$ avec passage en DEFAUT.
Résultat	aucune commande de mouvement tant que le paramètre n'est pas corrigé et acquitté.

Apport principal des essais

Le programme n'a pas seulement été parcouru sur son cycle nominal. Les valeurs limites et la saturation des huit bacs ont été provoquées volontairement, puis les corrections ont été rejouées sur le runtime local.

Limites et évolutions possibles

Distinguer ce qui est validé de ce qui dépend encore de la mécanique, de la Safety ou d'un axe réel.

Choix sur le point zéro

L'initialisation attend bZero. Quand le capteur est détecté, nBac prend la valeur 1 et le programme passe en AUTO. Aucune recherche mécanique du zéro n'est inventée, car son mouvement n'est pas défini dans le sujet.

Non implémenté dans cette version

- séquence mécanique complète de homing;
- fins de course du vérin;
- contrôle réel de position du plateau;
- timeout mécanique indépendant;
- chaîne d'arrêt d'urgence / Safety;
- diagnostic par code défaut détaillé;
- interface HMI.

Évolutions proposées

Priorité	Évolution
1	ajouter les capteurs de fin de course et séparer durée de commande et timeout de surveillance
1	intégrer une autorisation issue de la chaîne Safety et définir le comportement à la perte d'énergie
2	mémoriser un code défaut et l'étape d'apparition pour la maintenance
2	qualifier le filtrage du laser selon la cadence et le capteur réel
2	définir la conservation ou la remise à zéro des compteurs après coupure
3	ajouter une HMI simple : bac actif, compteurs, mode et cause de défaut
3	étudier la variante motorisée avec AXIS_REF et blocs PLCopen Motion

Variante motorisée

MC_Power, MC_MoveModulo, MC_Halt et MC_Stop restent des pistes d'évolution. Aucun de ces blocs n'est présenté comme utilisé ou testé dans la version pneumatique.

ANNEXE A · TWINCAT 3

Types et interface du bloc

Le source consolidé est joint séparément dans le fichier FB_Carrousel1.st.

Énumérations

```
1 {attribute 'qualified_only'}
2 {attribute 'strict'}
3 TYPE E_ModeCarrousel :
4 (
5     INIT      := 0,
6     AUTO      := 1,
7     DEFAULT   := 2
8 );
9 END_TYPE
10
11 {attribute 'qualified_only'}
12 {attribute 'strict'}
13 TYPE E_EtatCycle :
14 (
15     ATTENTE    := 0,
16     COMPTAGE   := 1,
17     BAC_PLEIN  := 2,
18     VERIN_SORTIE := 3,
19     VERIN_RENTREE := 4,
20     BAC_SUIVANT := 5,
21     ATTENTE_BAC := 6
22 );
23 END_TYPE
```

Lecture en ligne

Les énumérations rendent le mode et l'état courant directement compréhensibles dans TwinCAT, sans devoir interpréter des numéros d'étape.

Interface de FB_Carrousel

```
1 FUNCTION_BLOCK FB_Carrousel
2 VAR_INPUT
3     bLaser      : BOOL; // detection piece
4     bZero       : BOOL; // origine carrousel
5     nPieceBac   : INT;  // consigne / bac
6     tVerin      : TIME; // temps mouvement
7     aResetBac   : ARRAY[1..8] OF BOOL;
8 END_VAR
9
10 VAR_OUTPUT
11     bVerinAv    : BOOL; // cmd avant
12     bVerinAr    : BOOL; // cmd arriere
13     nBac        : INT;  // bac en position
14     bDefault    : BOOL; // default carrousel
15 END_VAR
16
17 VAR
18     eMode       : E_ModeCarrousel
19                 := E_ModeCarrousel.INIT;
20     eEtatCycle  : E_EtatCycle
21                 := E_EtatCycle.ATTENTE;
22     aPieceBac   : ARRAY[1..8] OF INT;
23     nImp        : INT := 0;
24     fbLaser     : R_TRIG;
25     fbTempoVerin : TON;
26     aTempoReset : ARRAY[1..8] OF TON;
27     bResetGlobal : BOOL;
28     i, j        : INT;
29 END_VAR
```

Organisation du source joint

Le fichier regroupe les deux DUTs, le bloc fonction et un exemple de MAIN. Dans le projet XAE, ces éléments restent répartis dans leurs objets TwinCAT respectifs.

10 · SYNTHÈSE

Conclusion

Une première logique simple a été transformée en un projet TwinCAT structuré, exécuté et éprouvé sur plusieurs scénarios.

Résultat obtenu

Le projet organise le carrousel autour de modes explicites et d'un cycle AUTO lisible. Il conserve huit compteurs, gère les resets, exécute quatre cycles de vérin et bloque proprement lorsqu'un bac déjà plein revient en position.

Ce que les essais ont apporté

- validation du chemin INIT → AUTO;
- observation du comptage sur front;
- contrôle de l'indexation et du bouclage 8 vers 1;
- validation des resets individuel et global;
- découverte puis correction de trois cas limites;
- acquittement défaut avec reprise par le zéro.

Lecture professionnelle du résultat

L'intérêt du travail ne repose pas sur l'idée d'un programme parfait dès la première écriture. La valeur vient de la démarche complète : **implémenter, exécuter, provoquer les limites, corriger et revalider.**

Dernier contrôle de configuration

Avant d'archiver le source consolidé comme version définitive : Rebuild, test rapide du zéro, d'un changement de bac, des deux resets et des deux paramètres invalides. Cette vérification concerne la mise au propre finale, pas les comportements déjà observés.

Message central

La solution simple présentée pendant le test a posé le principe. La reprise sous TwinCAT a ensuite permis de structurer le programme et de démontrer, par les essais, les corrections apportées aux situations non nominales.

RÉFÉRENCES TECHNIQUES

- Beckhoff Information System, [R_TRIG](#) - détection d'un front montant dans la bibliothèque Tc2_Standard.
- Beckhoff Information System, [exemple TwinCAT 3 associant machine d'états, temporisateur et trigger.](#)